

بسم الله الرحمن الرحيم

گزارش تمرین دوم سیستم‌های توزیع‌شده

سید محمد جزایری - 810101399

فهرست

1.....	فهرست
2.....	۱. معماری کلی سیستم و نقش ماشین‌های مجازی
2.....	جدول پیکربندی شبکه (آدرس‌های IP سرویس‌ها)
3.....	۲. لایه ارتباطی RPC و دلایل انتخاب فناوری‌ها
3.....	۲.۱. ارتباط VM1 و VM2 (سرویس احراز هویت): JSON-RPC
4.....	۲.۲. ارتباط VM1 و VM3 (سرویس فایل): Native Go RPC
5.....	۳. لایه داده و جداسازی اطلاعات کاربران (VM2)
7.....	۴. سازوکار Pub/Sub و پایش مصرف حافظه (بخش سوم)
7.....	۴.۱. پایش دوره ای حافظه
8.....	۴.۲. مدیریت آستانه و انتشار رخداد (Publisher)
9.....	۴.۳. دریافت رخداد و هشدار (Subscriber)
10.....	۴.۴. روش افزایش عمدی مصرف حافظه (Memory Leak Simulation)
10.....	۵. سناریوی تست عملیاتی و مستندات خروجی ترمینال
10.....	۵.۱. گام اول: ورود با اطلاعات نادرست و تایید خطا
11.....	۵.۲. گام دوم: ورود با اطلاعات درست و نمایش تصویر
12.....	۵.۳. گام سوم: تست پایش حافظه و دریافت هشدار Pub/Sub
14.....	۶. تحلیل فنی رفتار حافظه سیستم (Go Runtime Breakdown)
15.....	۷. چالش‌ها و محدودیت‌های پیاده‌سازی
15.....	۸. تصاویر تکمیلی

۱. معماری کلی سیستم و نقش ماشین‌های مجازی

این پروژه از یک معماری توزیع‌شده چند سرویسی (Multi-Service Architecture) با رویکرد لایه‌ای و کاملاً مستقل (Decoupled) تشکیل شده است. سیستم بر روی ۳ ماشین مجازی مجزا بر بستر لینوکس و یک پردازش به عنوان Subscriber مستقل در شبکه پیاده‌سازی شده است تا اصول جداسازی وظایف (Separation of Concerns) و عدم تمرکز به طور کامل رعایت شود.

- ماشین اول (VM1 - وب سرور): هسته تعامل با کاربر که یک سرور HTTP را میزبانی می‌کند. این ماشین فرم ورود را نمایش داده، فرآیند احراز هویت و دریافت فایل را مدیریت کرده و وظیفه پایش حافظه (Memory Monitoring) و انتشار رخدادها (Event Publishing) را بر عهده دارد.
- ماشین دوم (VM2 - سرویس احراز هویت): مخزن امن اطلاعات کاربران که هیچ‌گونه دسترسی مستقیمی از لایه وب به آن وجود ندارد. وظیفه آن صرفاً تایید یا رد اطلاعات ورود از طریق RPC است. همچنین پردازش Subscriber هم روی این ماشین قرار دارد و اطلاعات را از ماشین اول می‌گیرد.
- ماشین سوم (VM3 - سرویس فایل): یک مخزن ذخیره‌سازی ایزوله (Protected Storage) که تصاویر و اسناد سیستم را نگهداری کرده و تحت پروتکل RPC بومی به درخواست‌های تایید شده پاسخ می‌دهد.
- Subscriber مانیتورینگ (اجرا روی VM2/مستقل): یک پروسه کاملاً مجزا در شبکه که گوش خود را برای رخدادهای بحرانی حافظه وب‌سرور تیز کرده است.

جدول پیکربندی شبکه (آدرس‌های IP سرویس‌ها)

برای پایبندی به قوانین تمرین، از هیچ‌گونه آدرس localhost استفاده نشده و تمام ارتباطات بر بستر اینترفیس‌های واقعی شبکه Host-Only شکل گرفته‌اند:

نام ماشین / سرویس	آدرس IP شبکه	پورت ارتباطی	فناوری و پروتکل ارتباطی

VM1 (Web Server & Publisher)	192.168.56.103	8080	HTTP (رابط کاربری مرورگر)
VM2 (Auth Service)	192.168.56.104	50051	JSON-RPC over TCP
VM3 (File Service)	192.168.56.102	50052	Native Go RPC over TCP
Subscriber Node	192.168.56.104	50053	Go RPC (الگوی Pub/Sub)

۲. لایه ارتباطی RPC و دلایل انتخاب فناوری‌ها

۲.۱. ارتباط VM1 و VM2 (سرویس احراز هویت): JSON-RPC

برای بخش احراز هویت از فناوری JSON-RPC استفاده شده است.

- دلیل انتخاب: پروتکل JSON-RPC به دلیل استفاده از فرمت متنی سبک JSON، کاملاً مستقل از زبان برنامه نویسی (Language-Agnostic) است. این انتخاب تضمین می‌کند که اگر در آینده سرویس احراز هویت به زبان دیگری (مثل پایتون یا نودجی‌اس) بازنویسی شود، لایه وب‌سرور بدون کوچک‌ترین تغییری به کار خود ادامه دهد.

- Procedure پیاده‌سازی شده: متد AuthService.Login که فیلدهای Username و Password را به عنوان ورودی دریافت کرده و وضعیت منطقی موفقیت (Success) همراه با پیام سیستم (Message) را برمی‌گرداند.

```
// VM1 -> VM2 via JSON-RPC
conn2, err := net.Dial("tcp", VM2_ADDR)
if err != nil {
    http.Error(w, "Critical Error: Auth Service (VM2) is offline.", 500)
    return
}
defer conn2.Close()

client2 := jsonrpc.NewClient(conn2)

req := LoginRequest{
    Username: r.FormValue("username"),
    Password: r.FormValue("password"),
}
var res LoginResponse

err = client2.Call("AuthService.Login", req, &res)
if err != nil || !res.Success {
    w.Header().Set("Content-Type", "text/html")
    reason := res.Message
    if err != nil {
        reason = err.Error()
    }
    fmt.Fprintf(w, `<html><body style="text-align:center; color:red;">
        <h2>Login Failed!</h2>
        <p>Reason: %s</p>
        <a href="/login">Return to Login Page</a>
    </body></html>`, reason)
    return
}
```

در تصویر بالا ماشین اول سعی می کند که به ماشین دوم متصل شود اگر اتصال ناموفق بود به کاربر اعلام می کنیم ("Critical Error: Auth Service (VM2) is offline").

۲.۲. ارتباط VM1 و VM3 (سرویس فایل): Native Go RPC

برای جابه جایی فایل از سیستم Go RPC بومی (Native) استفاده شده است.

- دلیل انتخاب و دریافت فایل: از آنجا که جابه جایی فایل شامل داده های باینری و سنگین است، استفاده از ساختارهای متنی مثل REST یا JSON-RPC سرشار انکودینگ (Serialization Overhead) شدیدی ایجاد می کند. سیستم بومی Go RPC آرایه های بایت (byte[]) را به بهینه ترین شکل ممکن و با بالاترین سرعت در شبکه منتقل می کند. پس از دریافت این بایت ها در VM1،

داده‌ها به صورت Base64 در لایه وب انکود شده و درون تگ `img` به کاربر نمایش داده می‌شوند تا مسیر فیزیکی فایل روی VM3 کاملاً پنهان بماند.

- پروژر پیاده‌سازی شده: متد `FileService.GetFiles` که ساختار `FileRequest` (شامل نام فایل) را گرفته و محتوای بایت‌ها را در `FileResponse` تحویل می‌دهد.

```
// VM1 -> VM3 via normal Go RPC
client3, err := rpc.Dial("tcp", VM3_ADDR)
if err != nil {
    fmt.Fprintf(w, "<h1>Login Success!</h1><p>Welcome %s. (But VM3 is down for image fetch)</p>", req.Username)
    return
}
defer client3.Close()

fRes := FileResponse{}
err = client3.Call("FileService.GetFiles", FileRequest{"image.png"}, &fRes)

var imgBase64Str string
if err == nil {
    imgBase64Str = base64.StdEncoding.EncodeToString(fRes.Content)
}

w.Header().Set("Content-Type", "text/html")
fmt.Fprintf(w, `<html><body style="text-align:center;">
<h1 style="color:green;">Access Granted!</h1>
<p>Welcome, <b>%s</b></p>
<hr>
<h3>Protected Image from VM3:</h3>` +
func() string {
    if imgBase64Str != "" {
        return fmt.Sprintf(``, imgBase64Str)
    }
    return `<p style="color:red;">Failed to load image from VM3 storage.</p>`
}() +
`<br><br>
<a href="/login">Logout</a>
</body></html>`, req.Username)
```

مطابق تصویر بالا که ادامه تصویر قبلی است به محض اینکه احراز هویت از طرف ماشین اول با موفقیت انجام شد ماشین اول با ماشین سوم ارتباط برقرار می‌کند. اگر ماشین فعال نبود به کاربر این را اعلام می‌کنیم ("`h1>Login>`") در غیر اینصورت به ماشین سوم `RPC call` میکنیم و فایل تصویر را با انکودینگ Base64 دریافت می‌کنیم و برای کاربر به نمایش می‌گذاریم.

۳. لایه داده و جداسازی اطلاعات کاربران (VM2)

اطلاعات کاربران کاملاً از وب‌سرور مجزا شده و در فایل `users.json` واقع در سیستم فایل ماشین VM2 نگهداری می‌شود. وب‌سرور (VM1) هیچ‌گونه دسترسی

خواندن/نوشتن به این فایل ندارد و صرفاً از طریق فراخوانی تابع راه دور نتیجه را جویا می‌شود. ساختار داده ذخیره شده شامل اطلاعات کاربران تستی مطابق شکل زیر است:

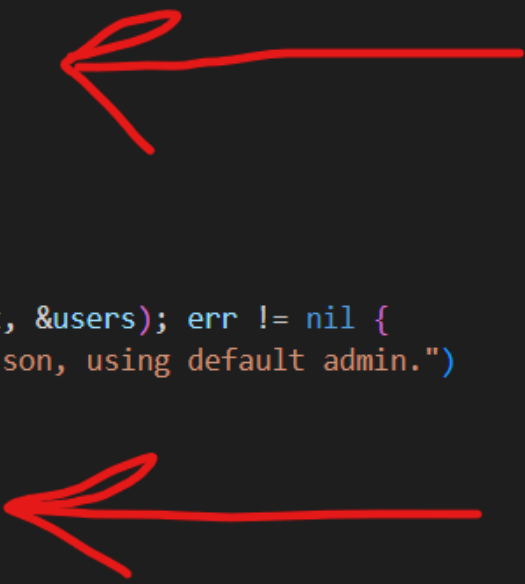
```
{ } users.json > ...  
1  {  
2    "alice": "alice123",  
3    "bob": "bob123",  
4    "admin": "admin123"  
5  }
```

نکته تحلیلی: برای پایداری سیستم (Fault Tolerance)، مکانیزم بارگذاری به گونه‌ای طراحی شده که در صورت بروز خطای ناگهانی در خواندن فایل، اطلاعات پیش‌فرض امن در حافظه لود شده تا سرویس‌دهی متوقف نشود. یعنی داده‌ها مطابق شکل

زیر در منطق برنامه hard-code شده‌اند:

```
func loadUsers() map[string]string {
    content, err := ioutil.ReadFile("users.json")
    if err != nil {
        log.Println("Error reading users.json, using default admin.")
        return map[string]string{
            "admin": "admin123",
            "alice": "alice123",
            "bob": "bob123",
        }
    }

    var users map[string]string
    if err := json.Unmarshal(content, &users); err != nil {
        log.Println("Invalid users.json, using default admin.")
        return map[string]string{
            "admin": "admin123",
            "alice": "alice123",
            "bob": "bob123",
        }
    }
    return users
}
```



البته واضح است که این کار تنها در این حالت که تعداد کاربران خیلی کم است قابل استفاده است و اگر تعداد کاربران بیشتر باشد scalable نیست.

۴. سازوکار Pub/Sub و پایش مصرف حافظه (بخش سوم)

۴.۱. پایش دوره ای حافظه

در داخل وب سرور (VM1)، یک گروتین (Goroutine) مستقل به عنوان Background Task در هر ۵ ثانیه با استفاده از تابع استاندارد runtime.ReadMemStats وضعیت حافظه فعال سیستم را رصد می‌کند. متریک مورد استفاده Alloc (میزان بایت‌های فعال تخصیص یافته در Heap) است.

```

var memoryHog [][]byte

func monitorMemory() {
    for {
        var m runtime.MemStats
        runtime.ReadMemStats(&m)
        memMB := m.Alloc / 1024 / 1024

        fmt.Printf("[Monitor] Current Memory: %d MB\n", memMB)

        thresholdMB := uint64(THRESHOLD / 1024 / 1024)

        if memMB >= thresholdMB {
            fmt.Printf("!!! ALERT: High Memory Usage: %d MB !!!\n", memMB)

            event := Event{
                EventType: "HIGH_MEMORY_USAGE",
                Service:    "web-server",
                MemoryMB:   memMB,
                ThresholdMB: thresholdMB,
                Timestamp:  time.Now().Format(time.RFC3339),
            }

            publish(event)
        }
        time.Sleep(5 * time.Second)
    }
}

```

۴.۲. مدیریت آستانه و انتشار رخداد (Publisher)

آستانه مجاز سیستم روی 300 MB تنظیم شده است که در تصویر بالا هم قابل مشاهده است. به محض اینکه مقدار $m.Alloc / 1024 / 1024$ با این آستانه برابر یا از آن بزرگتر شود، یک ساختار داده رخداد (Event Object) کاملاً منطبق بر فیلدهای درخواستی صورت تمرین تشکیل می‌شود:


```
type Event struct {
    EventType    string `json:"event_type"`
    Service      string `json:"service"`
    MemoryMB     uint64 `json:"memory_mb"`
    ThresholdMB  uint64 `json:"threshold_mb"`
    Timestamp    string `json:"timestamp"`
}
```

برای جلوگیری از مسدود شدن (Blocking) کدهای اصلی وب سرور و کاهش کارایی سیستم، فرآیند ارسال پیام به شبکه به صورت کاملاً آسنکرون به داخل یک گروتین مالتی ترید هدایت می شود (مطابق تصویر زیر).

```
func main() {
    go monitorMemory()

    http.HandleFunc("/login", loginHandler)
    http.HandleFunc("/consume-memory", consumeMemoryHandler)

    fmt.Println("Web Server running on :8080...")
    log.Fatal(http.ListenAndServe(":8080", nil))
}
```



۴.۳. دریافت رخداد و هشدار (Subscriber)

برنامه مستقل Subscriber روی پورت 50053 گوش می دهد. این سرویس پس از دریافت رخداد باینری از طریق شبکه، با استفاده از مکانیزم `json.MarshalIndent` داده ها را به ساختار متنی JSON زیبا (Pretty Print) تبدیل کرده و با یک ویژوالایزر واضح در ترمینال پرینت می کند. تصویر پایین کد این بخش است.

```
func (n *NotificationService) ReceiveEvent(event Event, reply *bool) error {
    eventJSON, err := json.MarshalIndent(event, "", " ")
    if err != nil {
        log.Printf("Error marshaling event: %v", err)
        return nil
    }

    log.Printf("\n--- [New Event Received] ---\n%s\n-----", string(eventJSON))

    *reply = true
    return nil
}
```

۴.۴. روش افزایش عمدی مصرف حافظه (Memory Leak Simulation)

یک Endpoint اختصاصی با آدرس /consume-memory ایجاد شده است. با هر بار فراخوانی این آدرس، یک آرایه بایت به بزرگی ۵۰ مگابایت تولید شده و به یک اسلایس سراسری (var memoryHog [][]byte) پیوند داده می‌شود. این پیوند سراسری مانع از پاکسازی حافظه توسط Garbage Collector زبان گو شده و به طور دقیق یک نشت حافظه پایدار را برای تست سناریو شبیه‌سازی می‌کند.

```
var memoryHog [][]byte
func consumeMemoryHandler(w http.ResponseWriter, r *http.Request) {
    chunk := make([]byte, 50*1024*1024)
    for i := range chunk {
        chunk[i] = 1
    }
    memoryHog = append(memoryHog, chunk)

    fmt.Fprintf(w, "Consumed 50MB. Simulated memory leak successfully.")
}
```

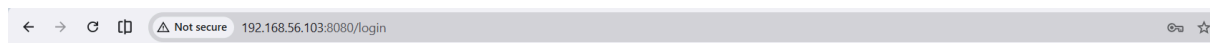
در این تصویر تابع و آرایه تعریف شده را مشاهده می‌کنیم.

۵. سناریوی تست عملیاتی و مستندات خروجی ترمینال

سیستم به طور کامل طبق سناریوی تست اجباری صورت تمرین با موفقیت اجرا شد:

۵.۱. گام اول: ورود با اطلاعات نادرست و تایید خطا

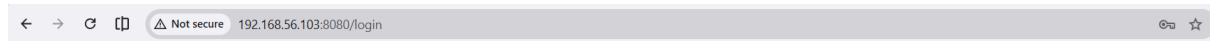
در صفحه وب ورود، کاربری با مشخصات اشتباه وارد شد. وب‌سرور درخواست را به VM2 فرستاد و با خطا روبرو شد:



Login to Distributed System

Username:

Password:



Login Failed!

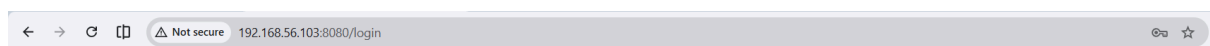
Reason: User not found

[Return to Login Page](#)

در تصاویر بالا به ترتیب صفحه لاگین , صفحه لاگین اشتباه را مشاهده می کنیم.

۵.۲. گام دوم: ورود با اطلاعات درست و نمایش تصویر

کاربر تستی alice با رمز صحیح وارد شد. فرآیند تایید در VM2 انجام شد:



Access Granted!

Welcome, alice

Protected Image from VM3:



[Logout](#)

```
mohammad@mohammad:~/DS_HW2/vm2_auth$ go run main.go
2026/05/26 23:32:00 Auth Service (VM2) is running on port 50051...
2026/05/26 23:32:22 [-] FAILED login: user='alice'
2026/05/26 23:32:42 [+] SUCCESS login: user='alice'
```

بلافاصله وب سرور با تایید این ورود، درخواستی را به سمت VM3 (سرور فایل) فرستاد تا تصویر داشبورد را لود کند:

```

mohammad@mohammad:~/DS_HW2/vm3_file$ go run main.go
2026/05/27 00:40:33 File RPC Service (VM3) is running on port 50052...
2026/05/27 00:41:27 RPC File Request: image.png
^Csignal: interrupt
mohammad@mohammad:~/DS_HW2/vm3_file$

```

نتیجه در مرورگر: صفحه موفقیت آمیز لود شد و تصویر دریافتی از VM3 بدون مشکل به کاربر نشان داده شد.

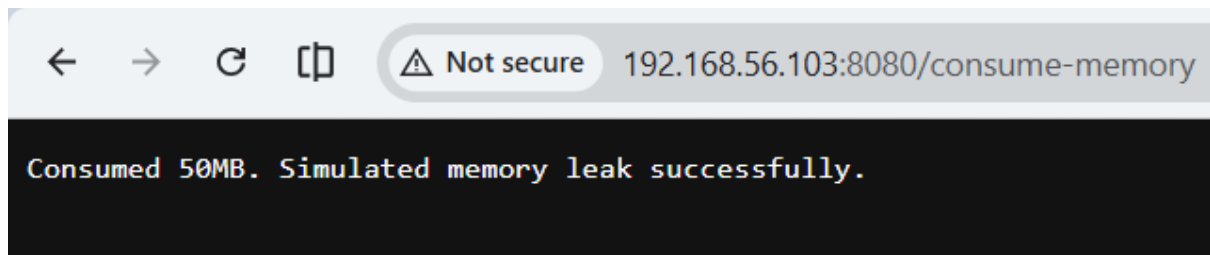
۵.۳. گام سوم: تست پایش حافظه و دریافت هشدار Pub/Sub

ابتدا Subscriber مانیتورینگ روی پورت ۵۰۰۵۳ فعال شد. در لایه وب سرور، مصرف حافظه در حالت عادی بین 0 MB تا 2 MB بود (افزایش ناچیز به ۲ مگابایت به دلیل Lazy Loading پکیج‌های شبکه و بیس ۶۴ در اولین درخواست کلاینت بود). با فراخوانی مکرر آدرس /consume-memory/ حافظه به صورت پله‌ای بالا رفت تا به حد مجاز رسید:

```

mohammad@mohammad:~/DS_HW2/vm1_web$ go run main.go
Web Server running on :8080...
[Monitor] Current Memory: 0 MB
[Monitor] Current Memory: 0 MB
[Monitor] Current Memory: 0 MB
[Monitor] Current Memory: 0 MB
[Monitor] Current Memory: 2 MB
[Monitor] Current Memory: 2 MB
[Monitor] Current Memory: 2 MB
[Monitor] Current Memory: 2 MB
[Monitor] Current Memory: 2 MB
[Monitor] Current Memory: 100 MB
[Monitor] Current Memory: 300 MB
[Monitor] Current Memory: 300 MB
[Monitor] Current Memory: 300 MB
[Monitor] Current Memory: 350 MB
!!! ALERT: High Memory Usage: 350 MB !!!
[Monitor] Current Memory: 350 MB
!!! ALERT: High Memory Usage: 350 MB !!!
[Monitor] Current Memory: 350 MB
!!! ALERT: High Memory Usage: 350 MB !!!
[Monitor] Current Memory: 350 MB
!!! ALERT: High Memory Usage: 350 MB !!!
[Monitor] Current Memory: 350 MB
!!! ALERT: High Memory Usage: 350 MB !!!
^Csignal: interrupt
mohammad@mohammad:~/DS_HW2/vm1_web$

```



تصویر بالا نمونه‌ای از فراخوانی consume-memory است و با هر بار reload پنجاه مگابایت دیگر تخصیص می‌یابد.

به محض عبور از مرز ۳۰۰ مگابایت، وب‌سرور هشدار را منتشر کرد و Subscriber مستقل رویداد را دقیقاً با ساختار درخواستی صورت سوال شکار و Pretty-Print کرد:

```

monammad@monammad:~/DS_HW2/vm2_auth$ go run subscriber.go
2026/05/27 00:40:29 Dedicated Monitoring Subscriber is listening on port 50053...
2026/05/27 00:42:13
--- [New Event Received] ---
{
  "event_type": "HIGH_MEMORY_USAGE",
  "service": "web-server",
  "memory_mb": 350,
  "threshold_mb": 300,
  "timestamp": "2026-05-27T00:42:13+03:30"
}
-----
2026/05/27 00:42:18
--- [New Event Received] ---
{
  "event_type": "HIGH_MEMORY_USAGE",
  "service": "web-server",
  "memory_mb": 350,
  "threshold_mb": 300,
  "timestamp": "2026-05-27T00:42:18+03:30"
}
-----
2026/05/27 00:42:23
--- [New Event Received] ---
{
  "event_type": "HIGH_MEMORY_USAGE",
  "service": "web-server",
  "memory_mb": 350,
  "threshold_mb": 300,
  "timestamp": "2026-05-27T00:42:23+03:30"
}
-----
2026/05/27 00:42:28
--- [New Event Received] ---
{
  "event_type": "HIGH_MEMORY_USAGE",
  "service": "web-server",
  "memory_mb": 350,
  "threshold_mb": 300,
  "timestamp": "2026-05-27T00:42:28+03:30"
}
-----
^Csignal: interrupt

```

۶. تحلیل فنی رفتار حافظه سیستم (Go Runtime Breakdown)

یک نکته برجسته در رفتار پایش حافظه این بود که پس از اولین فراخوانی لود حافظه، متریک به جای جمع ریاضی دقیق (مثلاً ۵۲ مگابایت)، عدد رند ۵۰ MB را نشان داد. تحلیل مهندسی این رفتار به دو علت است:

۱. بریدن اعشار در تقسیم صحیح (Integer Division): مقدار بایت‌های بازگردانده شده توسط Alloc بر عدد صحیح $1024 * 1024$ تقسیم می‌شود. چون متغیر از نوع uint64 است، بخش اعشاری کاملاً حذف (Truncate) می‌شود.

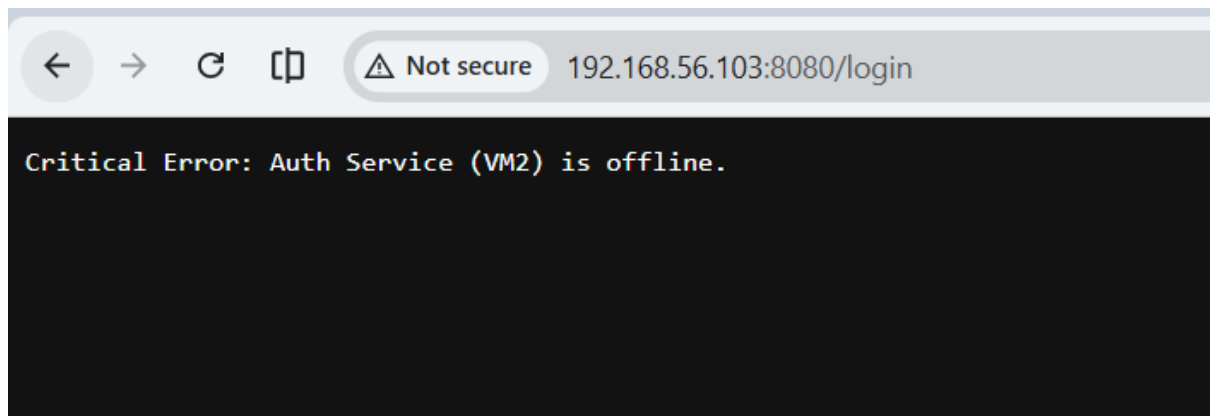
۲. رفتار Garbage Collector: ساختارهای داده موقتی که برای پاسخ‌دهی به پروتکل HTTP و تبدیل استریم‌های باینری ایجاد شده بودند، پس از پایان یافتن چرخه پاسخ‌دهی لایه وب فوراً توسط Garbage Collector به عنوان حافظه مرده شناسایی و پاکسازی شدند و حافظه فعال Heap دقیقاً روی مقدار بلوک سراسری ۵۰ مگابایتی فیکس شد.

۷. چالش‌ها و محدودیت‌های پیاده‌سازی

- سربر شبکه در مهاجرت داده‌های حجیم: جابه‌جایی فایل‌های بسیار بزرگ به صورت آرایه بایت یکپارچه (byte[]) در شبکه می‌تواند حافظه RAM و سرور را مسدود کند. راهکار آینده برای بهینه‌سازی این بخش، استفاده از مکانیزم gRPC Streaming یا ساختارهای Chunk-based بر بستر TCP برای جابه‌جایی تکه‌تکه تصاویر است. (اصلی دلیلی که در این تمرین از gRPC استفاده نکردم ناپایداری اینترنت و مشکلات اتصال به سرور go بود و حتی با استفاده از proxy که در تمرین قبلی در اختیارمان قرار گرفته بود هم موفق به دانلود پکیج‌های مناسب نشدم)
- اتصال‌های یکبار مصرف RPC: در طراحی فعلی، وب‌سرور برای هر درخواست لاگین یا دریافت فایل یک کانکشن TCP جدید می‌سازد و می‌بندد (Dial و Close). در سیستم‌های با مقیاس بالا (High Production)، این کار باعث اتمام پورت‌های آزاد ماشین (Ephemeral Port Exhaustion) می‌شود که باید با الگوی Connection Pooling یا بازنگه‌داشتن کانال‌های RPC رفع شود.

۸. تصاویر تکمیلی

در این بخش تصاویری از اتصال به ماشین اول با خاموش بودن ماشین دوم و اتصال به ماشین اول و احراز هویت موفق اما با خاموش بودن ماشین سوم می‌بینیم. همچنین تصاویری از ترمینال سه ماشین برای تشخیص آدرس IP هر کدام مشاهده می‌کنیم



Login Success!

Welcome bob. (But VM3 is down for image fetch)

```
mohammad@mohammad: ~/DS_HW2/vm3_file
^Csignal: interrupt
mohammad@mohammad:~/DS_HW2/vm3_file$ ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host
        valid_lft forever preferred_lft forever
2: enp0s3: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 08:00:27:f3:18:9c brd ff:ff:ff:ff:ff:ff
    inet 192.168.56.102/24 brd 192.168.56.255 scope global dynamic noprefixroute enp0s3
        valid_lft 497sec preferred_lft 497sec
    inet6 fe80::66f2:203a:8244:ac40/64 scope link noprefixroute
        valid_lft forever preferred_lft forever
3: enp0s8: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 08:00:27:fe:ed:41 brd ff:ff:ff:ff:ff:ff
    inet 10.0.3.15/24 brd 10.0.3.255 scope global dynamic noprefixroute enp0s8
        valid_lft 84197sec preferred_lft 84197sec
    inet6 fe80::3ce3:b4a4:bcde:7830/64 scope link noprefixroute
        valid_lft forever preferred_lft forever
```



```
mohammad@mohammad: ~/DS_HW2/vm2_auth
2026/05/26 23:32:42 [+] SUCCESS login: user='alice'
^Csignal: interrupt
mohammad@mohammad:~/DS_HW2/vm2_auth$ ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host
        valid_lft forever preferred_lft forever
2: enp0s3: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 08:00:27:f8:02:a8 brd ff:ff:ff:ff:ff:ff
    inet 192.168.56.104/24 brd 192.168.56.255 scope global dynamic noprefixroute enp0s3
        valid_lft 569sec preferred_lft 569sec
    inet6 fe80::ea3:ce6c:11cd:7f60/64 scope link noprefixroute
        valid_lft forever preferred_lft forever
3: docker0: <NO-CARRIER,BROADCAST,MULTICAST,UP> mtu 1500 qdisc noqueue state DOWN group default
    link/ether 02:42:1b:d1:f3:60 brd ff:ff:ff:ff:ff:ff
    inet 172.17.0.1/16 brd 172.17.255.255 scope global docker0
        valid_lft forever preferred_lft forever
mohammad@mohammad:~/DS_HW2/vm2_auth$
```

```
mohammad@mohammad: ~/DS_HW2/vm1_web
Web Server running on :8080...
^Csignal: interrupt
mohammad@mohammad:~/DS_HW2/vm1_web$ ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host
        valid_lft forever preferred_lft forever
2: enp0s3: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 08:00:27:48:db:54 brd ff:ff:ff:ff:ff:ff
    inet 192.168.56.103/24 brd 192.168.56.255 scope global dynamic noprefixroute enp0s3
        valid_lft 576sec preferred_lft 576sec
    inet6 fe80::b7ba:9ae9:a2a5:a14e/64 scope link noprefixroute
        valid_lft forever preferred_lft forever
3: docker0: <NO-CARRIER,BROADCAST,MULTICAST,UP> mtu 1500 qdisc noqueue state DOWN group default
    link/ether 02:42:3f:c3:ac:bf brd ff:ff:ff:ff:ff:ff
    inet 172.17.0.1/16 brd 172.17.255.255 scope global docker0
        valid_lft forever preferred_lft forever
mohammad@mohammad:~/DS_HW2/vm1_web$
```